MODEL_ALIASES_USER_GUIDE

Model Aliases User Guide

HelixCode Model Name Shortcuts

Version: 1.0 **Last Updated:** November 7, 2025 **Feature Status:** Production Ready

---

Table of Contents

Introduction

Model Aliases allow you to create short, memorable names for LLM models instead of typing long model identifiers. Instead of `gpt-4-turbo-preview` or `claude-3-opus-20240229`, you can simply use `gpt4-turbo` or `claude-opus`.

Benefits

- **Convenience:** Type less, work faster
- **Memorability:** Easy-to-remember names
- **Flexibility:** Create aliases that match your workflow
- **Fuzzy Matching:** Tolerates typos and variations
- **Multi-Provider:** Works with all 13 supported LLM providers

Supported Providers

Model aliases work with all HelixCode LLM providers:

Provider	Example Models
OpenAI	gpt-4, gpt-3.5-turbo, gpt-4-turbo-preview
Anthropic	claude-3-opus, claude-3-sonnet, claude-3-haiku
Google	gemini-pro, gemini-ultra, gemini-pro-vision
Ollama	llama-3-8b, mistral-7b, codestral-22b
Qwen	qwen-turbo, qwen-plus, qwen-max
xAI	grok-beta
Llama.cpp	(local models)
OpenRouter	(aggregated models)
AWS Bedrock	(AWS models)
Azure OpenAI	(Azure models)
Vertex AI	(Google Cloud models)
Groq	(Groq models)
Copilot	(GitHub Copilot models)

Quick Start

Using Built-in Aliases

HelixCode comes with sensible default aliases:

```
# Instead of this:
helix --model gpt-4-turbo-preview "Generate code"

# Use this:
helix --model gpt4-turbo "Generate code"

# Or even simpler:
helix --model gpt4 "Generate code"
```

Common Aliases

Alias	Target Model	Provider	Use Case
`gpt4`	gpt-4	OpenAI	Most capable
`gpt3`	gpt-3.5-turbo	OpenAI	Fast and cheap
`claude`	claude-3-opus	Anthropic	Large context
`gemini`	gemini-pro	Google	Multimodal
`llama`	llama-3-8b	Ollama	Local/privacy
`fast`	gpt-3.5-turbo	OpenAI	Quick tasks
`smart`	gpt-4	OpenAI	Complex tasks
`local`	llama-3-8b	Ollama	Offline work
`coding`	codestral-22b	Ollama	Code generation

Configuration

Configuration Files

Aliases are loaded from these locations (in order):

1. **Workspace:** `.helix/model-aliases.yaml`
2. **User:** `~/.config/helixcode/model-aliases.yaml`
3. **System:** `/etc/helixcode/model-aliases.yaml` (Linux)

Later files override earlier ones.

Basic Configuration Format

Create `.helix/model-aliases.yaml`:

```yaml
version: "1.0"
fuzzy_threshold: 0.7  # 70% similarity for fuzzy matching

aliases:
  - alias: gpt4
    target_model: gpt-4
    provider: openai
    description: GPT-4 (latest)
    tags: [openai, gpt, large]

  - alias: fast
```

```
    target_model: gpt-3.5-turbo
    provider: openai
    description: Fast and cheap
    tags: [fast, cheap]
```

Field Reference

Required Fields: - `alias`: The short name you'll use - `target_model`: The actual model identifier - `provider`: LLM provider name

Optional Fields: - `description`: Human-readable description - `tags`: Array of tags for searching/categorization

Example Configuration

See `config/model-aliases.example.yaml` for a complete example with 30+ pre-configured aliases.

Using Aliases

Command Line

```
# Use alias in command line
helix --model gpt4 "Write a function"

# Use alias in interactive mode
helix
> Use model: gpt4
> Generate a REST API
```

API

```
// Resolve alias in code
manager, _ := llm.LoadAliasManagerFromStandardPaths()
targetModel, provider, resolved := manager.Resolve("gpt4")
// targetModel = "gpt-4"
// provider = "openai"
// resolved = true
```

Chat Interface

```
# In chat, specify model with alias
@model gpt4
Please review this code...
```

Fuzzy Matching

Fuzzy matching allows typos and variations in alias names.

How It Works

The fuzzy matcher calculates similarity using: 1. **Exact match:** `gpt4` matches `gpt4` (100%) 2. **Contains match:** `gpt` matches `gpt4-turbo` (60%) 3. **Levenshtein distance:** `gpt4turbo` matches `gpt4-turbo` (92%)

Threshold

Default threshold: **0.7 (70% similarity)**

```
fuzzy_threshold: 0.7  # Adjust between 0.0 - 1.0
```

Lower threshold = more lenient (more matches, including incorrect) **Higher threshold** = more strict (fewer matches, more accurate)

Examples

```
# These all match "gpt4-turbo" (with default 0.7 threshold):
helix --model gpt4turbo      # ✓ 92% similarity
helix --model gpt-4-turbo    # ✓ Exact match
helix --model gpt4turb       # ✓ 85% similarity
helix --model gpt4trbo       # ✓ 75% similarity (typo)

# These don't match:
helix --model gpt            # ✗ Too short, ambiguous
helix --model turbo          # ✗ Different word
```

Disabling Fuzzy Matching

```
fuzzy_threshold: 1.0  # Require exact match
```

Managing Aliases

Adding Aliases

Via Configuration File:

```
aliases:
  - alias: my-model
```

```
    target_model: gpt-4
    provider: openai
    description: My preferred model
```

Via API:

```
manager := llm.NewAliasManager(0.7)
err := manager.AddAlias(&llm.ModelAlias{
    Alias:       "my-model",
    TargetModel: "gpt-4",
    Provider:    "openai",
})
```

Removing Aliases

```
err := manager.RemoveAlias("my-model")
```

Listing Aliases

```
# List all aliases
helix aliases list

# List by provider
helix aliases list --provider openai

# Search by tag
helix aliases search fast
```

API:

```
// List all
aliases := manager.ListAliases()

// List by provider
openaiAliases := manager.ListAliasesByProvider("openai")

// Search
matches := manager.SearchAliases("fast")
```

Import/Export

Export aliases:

```
helix aliases export > my-aliases.yaml
```

Import aliases:

```
helix aliases import my-aliases.yaml --overwrite
```

API:

```
// Export
aliases := manager.ExportAliases()

// Import
err := manager.ImportAliases(aliases, overwrite)
```

Advanced Features

Autocomplete

Get alias suggestions:

```
matches := manager.Autocomplete("gpt")
// Returns: ["gpt4", "gpt3", "gpt4-turbo", "gpt3-16k"]
```

Provider-Specific Resolution

When you have similar alias names, specify provider:

```
// If you have both "fast-openai" and "fast-anthropic"
model, provider, _ := manager.ResolveWithProvider("fast-openai",
        "openai")
```

Tag-Based Search

Find aliases by tags:

```
aliases:
  - alias: cheap-fast
    tags: [cheap, fast, openai]

  - alias: smart-slow
    tags: [smart, expensive, openai]
```

```
// Find all "fast" models
fastModels := manager.SearchAliases("fast")
```

Dynamic Threshold Adjustment

```
// Start strict
manager.SetFuzzyThreshold(0.9)
```

```
// Relax for user input
manager.SetFuzzyThreshold(0.6)
```

Best Practices

1. Use Descriptive Aliases

Good:

```
- alias: gpt4-turbo
- alias: claude-opus
- alias: llama-local
```

Avoid:

```
- alias: m1  # Not descriptive
- alias: a   # Too short
- alias: x   # Unclear
```

2. Follow Naming Conventions

Recommended patterns: - `<provider><version>`: `gpt4`, `claude3` - `<provider>-<variant>`: `gpt-turbo`, `claude-opus` - `<purpose>`: `fast`, `smart`, `coding`, `local` - `<purpose>-<provider>`: `fast-openai`, `coding-local`

3. Add Descriptions and Tags

```
- alias: coding
  target_model: codestral-22b
  provider: ollama
  description: Best model for code generation
  tags: [coding, local, mistral, specialized]
```

This makes aliases searchable and self-documenting.

4. Use Workspace-Specific Aliases

Create `.helix/model-aliases.yaml` in your project for project-specific models:

```
# Backend project
aliases:
  - alias: backend
    target_model: gpt-4
    description: Model for backend development
```

```yaml
# Frontend project
aliases:
  - alias: frontend
    target_model: claude-3-sonnet
    description: Model for frontend work
```

5. Set Appropriate Fuzzy Threshold

- **0.9-1.0:** Strict (production, scripts)
- **0.7-0.8:** Balanced (default, interactive use)
- **0.5-0.6:** Lenient (exploration, learning)

6. Organize by Use Case

```yaml
# Fast models
- alias: fast
- alias: quick
- alias: cheap

# Capable models
- alias: smart
- alias: capable
- alias: powerful

# Local models
- alias: local
- alias: offline
- alias: private

# Specialized
- alias: coding
- alias: vision
- alias: long-context
```

Examples

Example 1: Personal Workflow

```yaml
# ~/.config/helixcode/model-aliases.yaml
version: "1.0"
fuzzy_threshold: 0.7
```

```yaml
aliases:
  # My daily drivers
  - alias: work
    target_model: gpt-4
    provider: openai
    description: My work model
    tags: [work, daily]

  - alias: home
    target_model: llama-3-8b
    provider: ollama
    description: Home/offline model
    tags: [home, local, privacy]

  # Quick tasks
  - alias: quick
    target_model: gpt-3.5-turbo
    provider: openai
    tags: [fast, cheap]
```

Usage:

```
helix --model work "Review the PR"
helix --model home "Private conversation"
helix --model quick "Quick question"
```

Example 2: Team Configuration

```yaml
# Project: .helix/model-aliases.yaml
version: "1.0"

aliases:
  # Team standards
  - alias: review
    target_model: claude-3-opus
    provider: anthropic
    description: Model for code reviews (large context)

  - alias: generate
    target_model: gpt-4-turbo-preview
    provider: openai
    description: Model for code generation

  - alias: test
    target_model: gpt-3.5-turbo
```

```
    provider: openai
    description: Model for testing (cheap)
```

Team usage:

```
helix --model review "Review the authentication module"
helix --model generate "Generate API endpoints"
helix --model test "Generate test cases"
```

Example 3: Multi-Provider Setup

```
aliases:
  # OpenAI tiers
  - alias: openai-best
    target_model: gpt-4
    provider: openai

  - alias: openai-fast
    target_model: gpt-3.5-turbo
    provider: openai

  # Anthropic tiers
  - alias: anthropic-best
    target_model: claude-3-opus
    provider: anthropic

  - alias: anthropic-fast
    target_model: claude-3-haiku
    provider: anthropic

  # Google
  - alias: google-best
    target_model: gemini-pro
    provider: gemini

  # Local
  - alias: local-best
    target_model: llama-3-70b
    provider: ollama

  - alias: local-fast
    target_model: llama-3-8b
    provider: ollama
```

Example 4: Use-Case Based

```yaml
aliases:
  # By capability
  - alias: reasoning
    target_model: gpt-4
    provider: openai
    tags: [reasoning, logic]

  - alias: creative
    target_model: claude-3-opus
    provider: anthropic
    tags: [creative, writing]

  - alias: coding
    target_model: codestral-22b
    provider: ollama
    tags: [coding, programming]

  - alias: vision
    target_model: gemini-pro-vision
    provider: gemini
    tags: [vision, multimodal]

  # By context size
  - alias: long-context
    target_model: claude-3-opus  # 200k tokens
    provider: anthropic

  - alias: short-context
    target_model: gpt-3.5-turbo  # 4k tokens
    provider: openai
```

Troubleshooting

Alias Not Found

Problem:

```
Error: alias 'mymodel' not found
```

Solutions: 1. Check spelling: `helix aliases list` 2. Try fuzzy match: Use partial name 3. Add the alias to your config file

Wrong Model Resolved

Problem: Fuzzy match returns incorrect model

Solutions: 1. Increase threshold: `fuzzy_threshold: 0.9` 2. Use more specific alias name 3. Check for duplicate/similar aliases

Configuration Not Loaded

Problem: Aliases from config file not working

Solutions: 1. Check file location: `.helix/model-aliases.yaml` 2. Validate YAML syntax: `yamllint model-aliases.yaml` 3. Check file permissions: `chmod 644 model-aliases.yaml` 4. Verify HelixCode is looking in correct directory

Duplicate Alias Error

Problem:

```
Error: alias 'gpt4' already exists
```

Solutions: 1. Use different alias name: `gpt4-custom` 2. Remove existing alias first 3. Use `--overwrite` flag when importing

Provider Mismatch

Problem: Alias resolves to wrong provider

Solutions: 1. Specify provider in alias name: `fast-openai`, `fast-anthropic` 2. Use provider-specific resolution: `ResolveWithProvider()` 3. Check alias configuration has correct provider field

---

FAQ

Q: Can I use the same alias name for different providers? A: No, alias names must be unique globally. Use provider suffixes like `fast-openai` and `fast-anthropic`.

Q: What happens if I don't specify an alias? A: The model name is used as-is. Aliases are optional.

Q: Can I use aliases in API calls? A: Yes, resolve the alias first, then use the target model in your API call.

Q: How does fuzzy matching handle capitalization? A: Matching is case-insensitive. `GPT4`, `gpt4`, and `Gpt4` are all equivalent.

Q: Can I create aliases for non-existent models? A: Yes, but the model must exist when you try to use it.

Q: What's the performance impact of fuzzy matching? A: Minimal. Fuzzy matching adds <1ms for typical alias lists (<100 aliases).

Q: Can I disable fuzzy matching entirely? A: Yes, set `fuzzy_threshold: 1.0` for exact matches only.

Q: How do I share aliases with my team? A: Commit `.helix/model-aliases.yaml` to your repository.

Q: Can I have aliases in multiple files? A: Yes, HelixCode merges aliases from workspace, user, and system configs.

Q: What happens if there are duplicate aliases in different config files? A: Workspace config overrides user config, which overrides system config.

Q: Can I export aliases to share with others? A: Yes, use `helix aliases export` or copy your YAML file.

Q: How many aliases can I have? A: No hard limit, but keep it reasonable (<200) for performance.

See Also

- @ Mentions User Guide
- Slash Commands User Guide
- LLM Provider Configuration
- HelixCode Configuration Guide

Document Version: 1.0 **Last Updated:** November 7, 2025 **Feedback:** GitHub Issues